

EXPERIMENT-3

C Programming - Debugging Tools

- ★ **Name:** Anvika Gadde
- ★ **Sap ID-** 590033511
- ★ **Date:** 27/08/2026
- ★ **Subject:** C programming

AIM- To study different types of errors in C programming and understand various debugging techniques using printf(), compiler warnings, and GDB.

1. Fundamentals of Errors and Debugging Key Definitions

- **Error:** A mistake or blunder made by the programmer in the source code. Ex: Writing `a + b` when the intention was to multiply `a * b`.
- **Bug:** A flaw or fault in a computer program that causes it to produce an incorrect or unexpected result, or to behave in unintended ways. Ex: The program outputs the sum of two numbers instead of their product.
- **Debugging:** The systematic process of finding, analysing, and removing bugs or defects in a computer program.

2. Classifications of Errors in C

Error Type	Description	Primary Cause	Example
Syntax Errors	Code violates the grammatical rules of the C language.	Missing semicolons, unmatched braces, typos in keywords.	<code>int a = 10</code> (missing <code>;</code>)
Compilation / Type Errors	Operations performed on incompatible types or missing declarations.	Assigning float string to int pointer, using undeclared variable.	<code>int *p = "hello";</code>
Linker Errors	The compiler generates object files, but the linker	Function declared but not defined, missing libraries.	undefined reference to 'main'

	cannot combine them.		
Runtime Errors	The program crashes or terminates abnormally during execution.	Division by zero, accessing invalid memory addresses.	int x = 10 / 0;
Logical Errors	Code runs to completion without crashing, but output is incorrect.	Wrong mathematical formula, incorrect loop condition.	for(i=0; i<5; i++) instead of <=5

Why Does a Program Compile Successfully but Produce Incorrect Output?

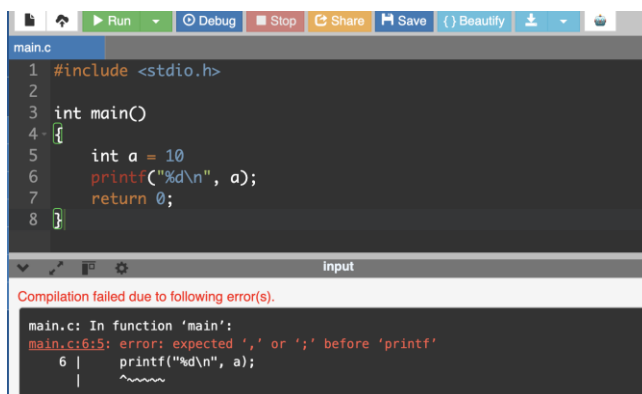
A compiler's job is to verify that your code obeys the syntax and type rules of the C language, and to convert that code into machine instructions. The compiler cannot understand human intent.

- If you write `sum = a - b;` instead of `sum = a + b;` the statement is completely valid C syntax. Therefore, compilation succeeds, but the resulting logic produces an erroneous output at execution time.

Case 1: Syntax & Compilation Errors

When the compiler encounters invalid code, it prints diagnostics specifying the file name, line number, column, and error description.

Example Code (sample1.c)



```

1 #include <stdio.h>
2
3 int main()
4 {
5     int a = 10
6     printf("%d\n", a);
7     return 0;
8 }

```

Compilation failed due to following error(s).

```

main.c: In function 'main':
main.c:6:5: error: expected ',' or ';' before 'printf'
6 |     printf("%d\n", a);
  |     ~~~~~

```

Breakdown of Error Message

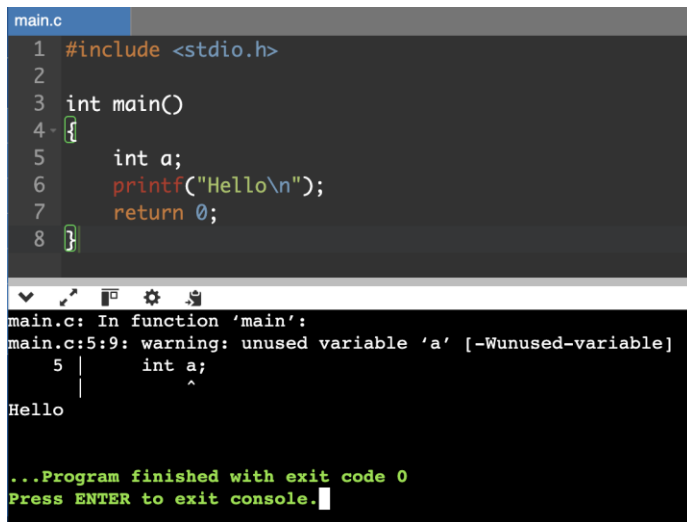
- Filename: sample1.c
- Line & Column: Line 5, Column 5

- Type of Message: error
- Description: Expected , or ; before printf statement.

Case 2: Compiler Warnings (-Wall, -Wextra)

A warning indicates code that is syntactically legal, but potentially dangerous or suspicious. Warnings do not stop compilation, but ignoring them frequently leads to bugs at runtime.

Example Code (sample2.c)



```
main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int a;
6      printf("Hello\n");
7      return 0;
8  }
```

```
main.c: In function 'main':
main.c:5:9: warning: unused variable 'a' [-Wunused-variable]
5 |     int a;
  |         ^
Hello

...Program finished with exit code 0
Press ENTER to exit console.
```

Standard Compilation vs. Warning Flags

Standard compilation (may suppress minor warnings)

```
$ cc sample2.c
```

```
$ ./a.out
```

```
Hello
```

Enabling comprehensive warning flags

```
$ cc -Wall -Wextra sample2.c
```

```
sample2.c:5:9: warning: unused variable 'a' [-Wunused-variable]
```

- **-Wall**: Enables a broad set of commonly useful compiler warnings.
- **-Wextra**: Enables additional warnings not covered by -Wall.

Case-3: printf-Based Debugging (Logical bug)

Before using complex debugging tools, simple program flow and state inspection can be performed using output statements (printf).

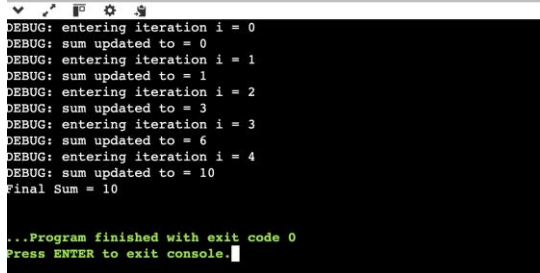
Basic Value Tracing

```
printf("DEBUG: a = %d, b = %d\n", a, b);
```

Example: Tracking Loop Execution (sample3.c)

- The following example demonstrates printf-based debugging by tracing the values of i and sum during each loop iteration. Debug printf() statements help observe the program flow and variable updates.

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int i, sum = 0;
6
7     for(i = 0; i < 5; i++)
8     {
9         printf("DEBUG: entering iteration i = %d\n", i);
10
11         sum = sum + i;
12
13         printf("DEBUG: sum updated to = %d\n", sum);
14     }
15
16     printf("Final Sum = %d\n", sum);
17
18     return 0;
19 }
```



```
DEBUG: entering iteration i = 0
DEBUG: sum updated to = 0
DEBUG: entering iteration i = 1
DEBUG: sum updated to = 1
DEBUG: entering iteration i = 2
DEBUG: sum updated to = 3
DEBUG: entering iteration i = 3
DEBUG: sum updated to = 6
DEBUG: entering iteration i = 4
DEBUG: sum updated to = 10
Final Sum = 10

...Program finished with exit code 0
Press ENTER to exit console.
```

3. Introduction to GDB (GNU Debugger)

Background: What is GNU & GDB?

- GNU**: A recursive acronym for "GNU's Not Unix!". It is an open-source operating system project initiated by Richard Stallman in 1983 to provide a completely free Unix-like system.
- GDB (GNU Debugger)**: The standard debugger for the GNU software system, allowing developers to inspect variable values, set execution breakpoints, and analyse crashes.

Installation & Compilation for GDB

- # Update package list and install GDB on Linux systems
\$ sudo apt update
\$ sudo apt install gdb
- # Compile source code with debugging symbols enabled (-g)
\$ cc -g -Wall -Wextra program.c -o program

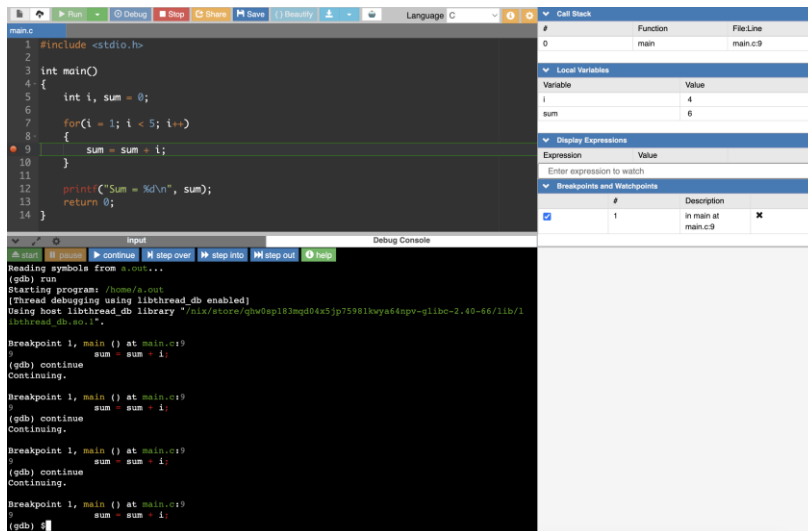
- # Launch GDB with the compiled binary
\$ gdb ./program

Quick Reference: Common GDB Commands

Command	Shorthand	Description / Purpose
Break <line/func>	b	Sets a breakpoint at a specified line number or function name
run	r	Starts execution of the program from the beginning.
next	n	Executes the next line of code (steps over function calls).
step	s	Executes the next line of code (steps into function calls).
print<var>	p	Prints the current value of a variable or expression.
display<var>	disp	Automatically prints the value of a variable after every step.
continue	c	Resumes execution until the next breakpoint or program completion
list	l	Prints source code lines around the current execution position.
info locals	info loc	Lists all local variables and their current values in the current frame
backtrace	bt	Displays the current call stack (chain of function calls).
info breakpoints	ib	Displays all active breakpoints and their numbers.
delete<id>	d	Deletes the specified breakpoint number (or all if omitted)
disable<id>	dis	Temporarily disables a breakpoint without deleting it.
Enable<id>	en	Re-enables a previously disabled breakpoint.
quit	q	Exits the GDB debugging session

Case 4: Debugging a Logical Error using GDB

Ex: Code with Logic Bug (sample4.c)



- Expected Output: 15 (1 + 2 + 3 + 4 + 5)
- Actual Output: 10 (1 + 2 + 3 + 4)

Debugging Walkthrough in GDB

```
$ cc -g sample4.c -o sample4
```

```
$ gdb ./sample4
```

```
(gdb) list
```

```
(gdb) break 7
```

```
Breakpoint 1 at 0x1149: file sample4.c, line 7.
```

```
(gdb) run
```

```
Starting program: ./sample4
```

```
Breakpoint 1, main () at sample4.c:7
```

```
7          sum = sum + i;
```

```
(gdb) print i
```

```
$1 = 1
```

```
(gdb) display i
```

```
1: i = 1
```

```
(gdb) display sum
```

```
2: sum = 0
```

```
(gdb) continue
```

```
Continuing.
```

```
Breakpoint 1, main () at sample4.c:7
```

```
7          sum = sum + i;
1: i = 2
2: sum = 1
```

(gdb) continue

```
...
1: i = 4
2: sum = 6
```

(gdb) continue

Sum = 10

[Inferior 1 (process) exited normally]

Conclusion: The loop terminates when i reaches 5 without executing the loop body for 5. The loop condition should be changed from $i < 5$ to $i \leq 5$.

Case 5: Debugging Functions (next vs. step)

- next: Executes the line. If the line contains a function call, the function executes entirely in the background without entering its body.
- step: Steps into the function call, transferring GDB's focus into the function body for line-by-line inspection.

Example Code (sample5.c)

The screenshot displays the GDB interface for debugging a C program named `sample5.c`. The main window shows the source code with a breakpoint set at line 13. The right sidebar shows the Local Variables table with `result1` set to 25, and the Breakpoints and Watchpoints table with a breakpoint at line 2. The bottom panel shows the GDB console output.

```
main.c
1  #include <stdio.h>
2
3  int square(int x)
4  {
5      int result1;
6      result1 = x * x;
7      return result1;
8  }
9
10 int main()
11 {
12     int result, n = 5;
13     result = square(n);
14     printf("Square = %d\n", result);
15     return 0;
16 }
```

Local Variables

Variable	Value
result1	25

Display Expressions

Expression	Value
Enter expression to watch	

Breakpoints and Watchpoints

#	Description	File:Line
2	in main at main.c:13	main.c:13

Debug Console

```
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/ghw0sp183mqd04x5jp75981kwya64npv-glibc-2.40-66/lib/libthread_db.so.1".

Breakpoint 2, main () at main.c:13
13     result = square(n);
(gdb) step
square (x=5) at main.c:6
6     result1 = x * x;
(gdb) print x
$1 = 5
(gdb) next
7     return result1;
(gdb) print result
'result' has unknown type; cast it to its declared type
(gdb) print result1
$2 = 25
(gdb)
```

Option A: Stepping Over with next

```
(gdb) break main
(gdb) run
(gdb) next          # Moves to result = square(n);
(gdb) next          # Executes square(n) completely
(gdb) print result
$1 = 25             # Result obtained without stepping inside square()
```

Option B: Stepping Into with step

```
(gdb) break main
(gdb) run
(gdb) next          # Moves to result = square(n);
(gdb) step          # Enters square(x) function body!
square (x=5) at sample5.c:5
5      result1 = x * x;
(gdb) print x
$1 = 5
(gdb) next
(gdb) print result1
$2 = 25
```

Case 6: Debugging Runtime Errors using Backtrace (backtrace / bt)

When a program crashes (e.g., floating point exception / division by zero), GDB can pinpoint the exact function call sequence leading to the crash.

Example Code (sample6.c)

```
main.c
1
2
3 int divide(int a, int b)
4 {
5     return a / b;
6 }
7
8 int calculate(int x)
9 {
10    return divide(x, 0);
11 }
12
13 int main()
14 {
15     int result = calculate(10);
16     printf("Result: %d\n", result);
17     return 0;
18 }
```

#	Function	File:Line
0	divide	main.c:5
1	calculate	main.c:10
2	main	main.c:15

```

(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/ghw0sp183mqd04x5jp75981kwy64nnpv-glibc-2.40-66/lib/libthread_db.so.1".

Program received signal SIGFPE, Arithmetic exception.
0x000055555555147 in divide (a=10, b=0) at main.c:5
5      return a / b;
(gdb) backtrace
#0  0x000055555555147 in divide (a=10, b=0) at main.c:5
#1  0x000055555555166 in calculate (x=10) at main.c:10
#2  0x00005555555517a in main () at main.c:15
(gdb) $
```

Local Variables	
Variable	Value

Display Expressions	
Expression	Value
Enter expression to watch	

Breakpoints and Watchpoints	
#	Description

GDB Backtrace Execution

```
$ cc -g sample6.c -o sample6
$ gdb ./sample6
(gdb) run
Program received signal SIGFPE, Arithmetic exception.
0x0000555555555132 in divide (a=10, b=0) at sample6.c:5
5          return a / b;
(gdb) backtrace
#0 0x0000555555555132 in divide (a=10, b=0) at sample6.c:5
#1 0x0000555555555151 in calculate (x=10) at sample6.c:10
#2 0x000055555555516a in main () at sample6.c:15
```

Understanding Stack Frames

- Frame #0: The function where the crash actually occurred (divide).
- Frame #1: The function that called frame #0 (calculate).
- Frame #2: The top-level function that started the call sequence (main).

You can inspect specific frames using the frame command:

```
(gdb) frame 0
(gdb) print b          # Shows b = 0, identifying division by zero
(gdb) frame 1
(gdb) print x          # Shows x = 10
```

Case 7: Debugging Array Out-of-Bounds Errors using GDB

In C, array bounds are not checked at runtime automatically. Accessing indices beyond the declared limit results in undefined behaviour or memory corruption.

[Example Code \(sample7.c\)](#)

The screenshot shows the GDB interface with a C program named `main.c`. The program defines an array `a` of size 5 and a loop that reads input into `a[i]` until `i=5`. A breakpoint is set at line 10. The GDB console shows the program running and the breakpoint hitting at line 10, where `scanf` is called. The local variables panel shows `a` as an array of 5 integers and `i` as 0. The breakpoints panel shows a breakpoint at line 10.

Inspection in GDB

(gdb) break 8

(gdb) run

(gdb) print &a[0]

\$1 = (int *) 0x7ffffffd9c0

(gdb) print &a[4]

\$2 = (int *) 0x7ffffffd9d0

(gdb) print &a[5]

\$3 = (int *) 0x7ffffffd9d4

of bounds!

Valid array size is 5 (indices 0..4). Index 5 is out

- Correct Loop Condition: `for(i = 0; i < 5; i++)`

Case 8: Segmentation Faults (SIGSEGV) using GDB

A Segmentation Fault occurs when a program attempts to access a memory location that it does not have permission to access (such as dereferencing a NULL pointer).

```
main.c
1 #include <stdio.h>
2
3 int main()
4 {
5     int *p = NULL;
6
7     *p = 10;
8
9     printf("%d\n", *p);
10    return 0;
11 }
```

#	Function	File:Line
0	main	main.c:7

Local Variables	
Variable	Value
p	0x0

Display Expressions	
Expression	Value
Enter expression to watch	

Breakpoints and Watchpoints	
#	Description

Input: start, pause, continue, step over, step into, step out, help

Debug Console

```
Reading symbols from a.out...
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/ghw0sp183mqd04x5jp75981krya64npv-glibc-2.40-66/lib/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x000055555555514d in main () at main.c:7
7      *p = 10;
(gdb) print p
$1 = (int *) 0x0
(gdb) print *p
Cannot access memory at address 0x0
(gdb)
```

Debugging in GDB

```
$ cc -g sample8.c -o sample8
```

```
$ gdb ./sample8
```

```
(gdb) run
```

```
Program received signal SIGSEGV, Segmentation fault.
```

```
0x0000555555555139 in main () at sample8.c:6
```

```
6 *p = 10;
```

```
(gdb) print p
```

```
$1 = (int *) 0x0
```

```
(gdb) print *p
```

```
Cannot access memory at address 0x0
```

4. Systematic Debugging Strategy

To debug effectively, follow this 8-step methodology:

1. Observe Expected vs. Actual Output
2. Reproduce the Issue Consistently
3. Compile with Flags (-Wall -Wextra -g)
4. Isolate Suspicious Code Sections
5. Inspect State (printf or GDB breakpoints)

6. Apply a Single Minimal Fix
7. Recompile and Re-test
8. Verify with Edge Cases & Boundary Input

5. Exercises & Homework Questions

1. Error Classification: Write a small C program snippet illustrating each of the following error types: Syntax error, Compilation/Type error, Linker error, Runtime error, and Logical error.

1.Syntax Error

Occurs when the grammatical rules of C are violated.

```
int a =10  
printf(“%d”,a);
```

Error: Missing semicolon(;)

2.Compilation/Type Error

Occurs when incompatible data types are used or declarations are incorrect.

```
Int *p= “hello”;
```

Error: Incompatible pointer type.

3.Linker Error

- Occurs when a function is declared/called but its definition is unavailable during linking.

```
void display ( );  
int main ( )  
{  
  
    display( );  
    return0;  
}
```

Error: display () is declared but not defined ,resulting in an undefined reference.

4.Runtime Error

- Occurs while the program is executing
int a=10;
int b=0;

```
int x=a/b;
```

Error: Division by zero

5.Logical Error

- The program compiles and executes, but produces the wrong result.

```
int a =10, b=5;
```

```
int sum = a -b ;    // should be a+b
```

Error: wrong operation is used, so the output is incorrect.

2. GNU Concept: Explain what the GNU project is and how GDB fits into the GNU ecosystem

GNU stands for “**GNU’s Not Unix!**”. it is an open source operating system project started by Richard Stallman in 1983 with the aim of providing a completely free Unix like system.

GDB(GNU Debugger) is the standard debugger of the GNU software system. It helps programmers debug programs by setting breakpoints, inspecting variable values, stepping through code, and analysing program crashes.

IN short: GNU provides the free software ecosystem, and GDB is its debugging tool.

3. Shell Context: Define what a Shell and Kernel are. Briefly list three common Linux shells (e.g., bash, zsh, fish).

Shell: A shell is a command line interface that allows the user to interact with the operating system by entering commands.

Kernel: the kernel is the core part of an operating system, it manages hardware resources such as the CPU, memory ,devices ,and processes, and acts as a bridge between software and hardware.

Three common Linux shells:

1.Bash - Bourne again Shell

2.Zsh - Z shell

3. Fish - Friendly Interactive Shell

4. Research and contrast the differences between: Breakpoint, Watchpoint, Catchpoint, and Tracepoint.

Debugging Point	Meaning
Breakpoint	Pauses program execution when a specific line or function is reached

Watchpoint	Pauses execution when the value of a variable or memory location changes.
Catchpoint	Pauses execution when a specific event occurs, such as a signal ,exception ,or system call.
Tracepoint	Collects information at a specified point without normally stopping program execution, useful for observing program behaviour.